

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	PRUEBAS DE SOFTWARE				
TÍTULO DE LA PRÁCTICA:	Depuración y Pruebas de Software				
NÚMERO DE PRÁCTICA:	01	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	4	
FECHA INICIO:	13/04/2026	FECHA FIN:	17/04/2026	DURACIÓN:	100 min
RECURSOS A UTILIZAR: ● Una computadora personal ● Lenguaje de programación Python ● Visual Studio Code					
DOCENTE(s): ● Prof. Robert Arisaca / Prof. Diego Iquira / Prof. Lino Pinto					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: ● El estudiante comprenderá y analizará la diferencia entre actividades de depuración y el proceso de pruebas de software ● El estudiante elaborará casos de pruebas para demostrar que un programa computacional cumple una especificación funcional.	
TEMAS: ● Depuración versus Pruebas de Software ● Diseño de Casos de Prueba ● Validación de casos de prueba	
COMPETENCIAS	C.m Construye responsablemente soluciones siguiendo un proceso adecuado llevando a cabo las pruebas ajustadas a los recursos disponibles del cliente.

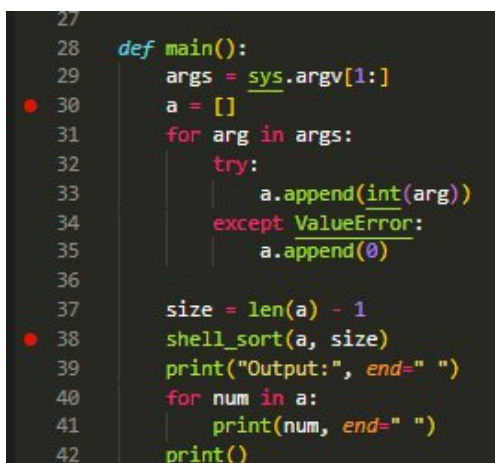
	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 2

CONTENIDO DE LA GUÍA

● MARCO CONCEPTUAL

¿Qué es la Depuración?

La **depuración** (debugging) es el proceso mediante el cual se identifican, analizan y corrigen errores (bugs) en el código fuente de un software. Estos errores pueden ir desde problemas de lógica, manejo incorrecto de datos, errores en la sintaxis o problemas de rendimiento. La depuración es una actividad esencial durante el ciclo de desarrollo, ya que ayuda a mejorar la calidad del código y la estabilidad del sistema.



```

27
28 def main():
29     args = sys.argv[1:]
30     a = []
31     for arg in args:
32         try:
33             a.append(int(arg))
34         except ValueError:
35             a.append(0)
36
37     size = len(a) - 1
38     shell_sort(a, size)
39     print("Output:", end=" ")
40     for num in a:
41         print(num, end=" ")
42     print()

```

Figura 1 – Breakpoint condicional en Visual Studio Code

Actividades clave en la depuración

- **Identificación del error:** Esto puede ocurrir cuando se detecta un comportamiento no esperado en la aplicación. Por ejemplo, al recibir un mensaje de error o al notar que una funcionalidad no responde como se esperaba.
- **Reproducción del error:** Es fundamental poder replicar el problema en un entorno controlado para entender bajo qué condiciones ocurre.
- **Uso de herramientas de depuración:** Utilizar herramientas como depuradores integrados en los IDE (p. ej., Visual Studio Code, IntelliJ o Eclipse), que permiten establecer puntos de interrupción (breakpoints), inspeccionar variables y seguir la ejecución del programa paso a paso.
- **Análisis del Código:** Revisar el flujo de ejecución y la lógica para aislar la causa raíz del error.
- **Corrección y Validación:** Modificar el código para corregir el error y validar que la solución funciona sin introducir nuevos problemas.

Ejercicio 1.1

- Abre Visual Studio Code y crea un nuevo proyecto “/LAB_01/” y luego cree un nuevo archivo de código con el nombre “shell_sort.py”, pega el siguiente código:

```
# Algoritmo: Shell Sort mejorado (Donald Knuth, 1973)
# Recibe números como argumentos desde la línea de comandos,
# los ordena usando el algoritmo Shell Sort y los imprime ordenados.

import sys

def shell_sort(a, size):
    h = 1
    while h <= size // 3:
        h = h * 3 + 1
    while h != 1:
        h //= 3
        for i in range(h, size):
            v = a[i]
            j = i
            while j >= h and a[j - h] > v:
                a[j] = a[j - h]
                j -= h
            if i != j:
                a[j] = v

# Es una mejora del Insertion Sort clásico. En vez de mover elementos de uno en uno,
# los mueve en saltos grandes primero, lo que lo hace significativamente
# más eficiente con listas grandes.

def main():
    args = sys.argv[1:]
    a = []
    for arg in args:
        try:
            a.append(int(arg))
        except ValueError:
            a.append(0)

    size = len(a) - 1
    shell_sort(a, size)
    print("Output:", end=" ")
    for num in a:
        print(num, end=" ")
    print()

if __name__ == "__main__":
    main()
```

- Compila y ejecuta el programa con estos argumentos:

> python shell_sort.py 9 7 3 11

> output: 3 7 9 11

SALIDA CORRECTA

Ahora ejecuta el programa con otros argumentos:

> python shell_sort.py 14 2 7 1

> output: 2 7 14 1

SALIDA FALLIDA

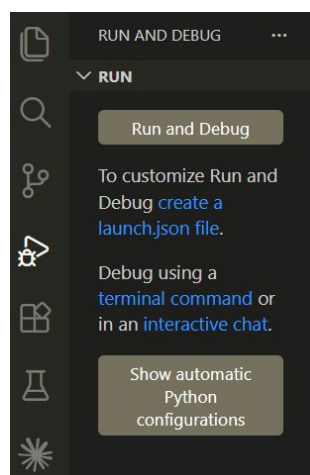
> python shell_sort.py 14 \$ 7 A

> output: 0 7 14 0

SALIDA FALLIDA

¿Cuál es el error en el programa?

- Depura el programa con ayuda del docente hasta encontrar el error, activa la opción “Run a Debug” en VS Code:



- Haz clic en "create a launch.json file"
- Selecciona **Python Debugger** → **Python File**
- Se abrirá el archivo .vscode/launch.json. Reemplaza su contenido con esto:

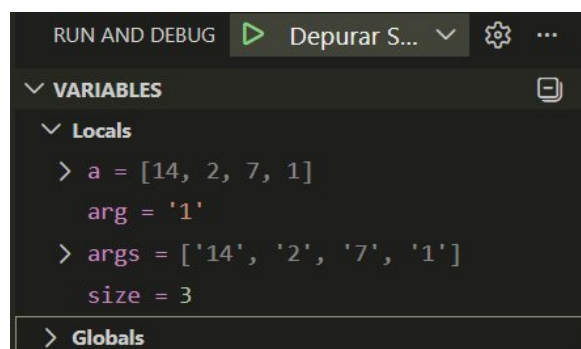
```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Depurar Shell Sort",
      "type": "debugpy",
      "request": "launch",
      "program": "${workspaceFolder}/shell_sort.py",
      "args": ["14", "2", "7", "1"],
      "console": "integratedTerminal"
    }
  ]
}
```

El campo "args" es donde pones tus datos: cada número como string separado.

- Abre `shell_sort.py` y haz clic en el **margen izquierdo** (junto al número de línea) para colocar breakpoints. Aparecerá un **círculo rojo** ●

```
28 def main():
29     args = sys.argv[1:]
30     a = []
31     for arg in args:
32         try:
33             a.append(int(arg))
34         except ValueError:
35             a.append(0)
36
37     size = len(a) - 1
38     shell_sort(a, size)
39     print("Output:", end=" ")
40     for num in a:
41         print(num, end=" ")
42     print()
43
```

- Presiona [F5] para iniciar el proceso de depuración, se mostrará en la parte superior izquierda el listado de variables y sus valores respectivo, corrige el error.



- En el programa `shell_sort.py` se ha detectado en la línea 37 que el tamaño del vector (`size`) no corresponde al número de argumentos, el programador por error introdujo el error: `size = len(a) - 1`, la instrucción correcta es `size = len(a)`
- Controles de depuración en VS Code:

Botón	Tecla	Qué hace
▶ Continue	F5	Continúa hasta el siguiente breakpoint
↪ Step Over	F10	Ejecuta la línea actual sin entrar a funciones
↓ Step Into	F11	Entra dentro de una función
↑ Step Out	Shift+F11	Sale de la función actual
↺ Restart	Ctrl+Shift+F5	Reinicia la depuración
■ Stop	Shift+F5	Detiene la depuración

¿Qué es Probar Software?

Las pruebas (testing) de software son procesos sistemáticos diseñados para evaluar y validar que una aplicación cumple con los requisitos funcionales y no funcionales especificados. Mientras la depuración se centra en encontrar y corregir errores, las pruebas buscan asegurar que el sistema se comporte de la forma esperada en diferentes escenarios, ayudando a prevenir la introducción de nuevos errores a medida que el software evoluciona.

Conceptos clave en las pruebas:

- **Error:** una acción humana que produce un resultado incorrecto.
- **Defecto (*fault o bug*):** un desperfecto que puede causar que el componente o el sistema falle al realizar su función requerida. Por ejemplo, una sentencia incorrecta o una definición incorrecta de los datos. Si es encontrado un defecto durante la ejecución, puede causar una falla del componente o sistema.
- **Falla (*failure*):** desviación del componente o el sistema de su prevista entrega, servicio o resultado.

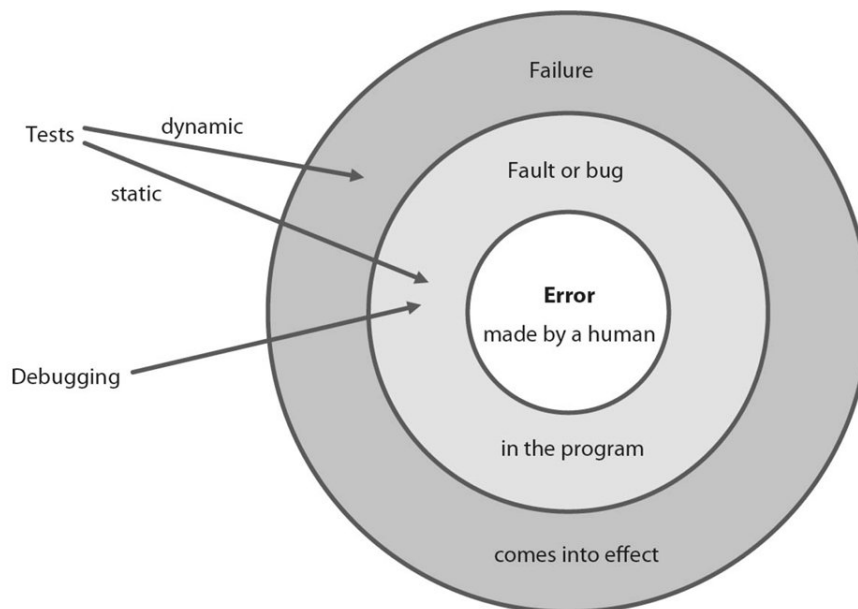


Figura 2 – Relación entre error, defecto y falla (Spillner & Linz, 2021)

¿Qué es un Caso de Prueba?

Un **caso de prueba** es un conjunto de condiciones, entradas, acciones y resultados esperados que se utilizan para determinar si una parte específica de un software funciona correctamente. Cada caso de prueba está diseñado para verificar que una función, módulo o componente se comporte de acuerdo con los requisitos especificados. En esencia, es una descripción detallada de cómo probar una funcionalidad particular del sistema.

Componentes de un Caso de Prueba

Un caso de prueba generalmente incluye los siguientes elementos:

- **Identificador:** Un código o número único que permite referenciar el caso de prueba.
- **Descripción:** Una breve explicación sobre qué funcionalidad o escenario se va a probar.
- **Datos de Entrada:** Los valores o condiciones que se utilizarán durante la prueba.
- **Resultado Esperado:** La salida o comportamiento que se espera después de ejecutar la prueba.
- **Resultado Real (opcional):** El resultado obtenido tras la ejecución, que se utiliza para comparar con lo esperado y determinar si la prueba pasó o falló.

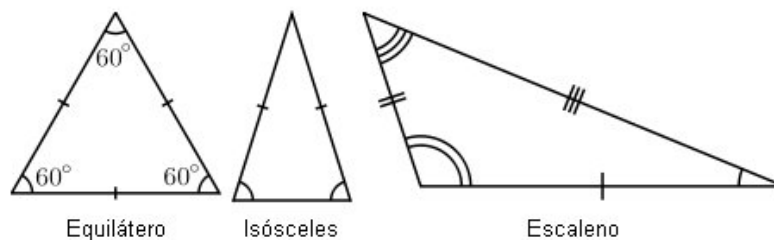
	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 8

Ejercicio 1.2 - Analice el siguiente enunciado (Myers, 2012):

“El programa lee 3 enteros. Estos enteros se interpretan como la longitud de los lados de un triángulo. El programa imprime un mensaje que muestra si el triángulo es escaleno, isósceles, equilátero o un triángulo inválido.”

Recuerde que:

- Un triángulo es equilátero si sus tres lados tienen la misma longitud (los tres ángulos internos miden 60 grados).
- Un triángulo es isósceles si tiene dos lados de la misma longitud. Los ángulos que se oponen a estos lados tienen la misma medida.
- Un triángulo es escaleno si todos sus lados tienen longitudes diferentes. En un triángulo escaleno no hay ángulos con la misma medida.



1. Inicie Visual Studio Code, apertura el proyecto “/LAB_01/” y luego cree un nuevo archivo de código con el nombre “triangle.py” y escriba el siguiente código Python:

```
def clasificar_triángulo(lado1, lado2, lado3):
    """
    Esta función clasifica un triángulo según sus lados.

    Args:
        lado1: La longitud del primer lado.
        lado2: La longitud del segundo lado.
        lado3: La longitud del tercer lado.

    Returns:
        Un string que indica si el triángulo es escaleno, isósceles, equilátero
        o inválido.
    """

    # Verificar si el triángulo es válido
    if lado1 + lado2 <= lado3 or lado1 + lado3 <= lado2 or lado2 + lado3 <=
lado1:
        return "Triángulo inválido"

    # Clasificar el triángulo
```



```
if lado1 == lado2 == lado3:
    return "Triángulo equilátero"
elif lado1 == lado2 or lado1 == lado3 or lado2 == lado3:
    return "Triángulo isósceles"
else:
    return "Triángulo escaleno"

# Solicitar al usuario la longitud de los lados
lado1 = int(input("Ingrese la longitud del primer lado: "))
lado2 = int(input("Ingrese la longitud del segundo lado: "))
lado3 = int(input("Ingrese la longitud del tercer lado: "))

# Clasificar el triángulo y mostrar el resultado
clasificacion = clasificar_trianguulo(lado1, lado2, lado3)
print(f"El triángulo es: {clasificacion}")
```

2. Ingrese valores inválidos que hagan fallar el programa, documente cada falla como un caso de prueba.
3. Para que el programa sea aceptado funcionalmente (validado) debe pasar los siguientes casos de prueba:
 - 1) ¿Tienes un caso de prueba que represente un triángulo escaleno válido? (Ten en cuenta que casos de prueba como 1, 2, 3 y 2, 5, 10 no justifican una respuesta afirmativa porque un triángulo con esas dimensiones no es válido.)
 - 2) ¿Tienes un caso de prueba que represente un triángulo equilátero válido?
 - 3) ¿Tienes un caso de prueba que represente un triángulo isósceles válido? (Ten en cuenta que un caso de prueba que represente 2, 2, 4 no contaría porque no es un triángulo válido.)
 - 4) ¿Tienes al menos tres casos de prueba que representen triángulos isósceles válidos, de manera que hayas probado las tres permutaciones de dos lados iguales (tales como 3, 3, 4; 3, 4, 3; y 4, 3, 3)?
 - 5) ¿Tienes un caso de prueba en el que un lado tenga valor cero?
 - 6) ¿Tienes un caso de prueba en el que un lado tenga un valor negativo?
 - 7) ¿Tienes un caso de prueba con tres enteros mayores que cero tal que la suma de dos de los números sea igual al tercero? (Es decir, si el programa dijera que 1, 2, 3 representa un triángulo escaleno, contendría un error/bug.)
 - 8) ¿Tienes al menos tres casos de prueba en la categoría 7, de manera que hayas probado las tres permutaciones donde la longitud de un lado es igual a la suma de las longitudes de los otros dos lados (p. ej., 1, 2, 3; 1, 3, 2; y 3, 1, 2)?
 - 9) ¿Tienes un caso de prueba con tres enteros mayores que cero tal que la suma de dos de los números sea menor que el tercero (tales como 1, 2, 4 o 12, 15, 30)?
 - 10) ¿Tienes al menos tres casos de prueba en la categoría 9, de manera que hayas probado las tres permutaciones (p. ej., 1, 2, 4; 1, 4, 2; y 4, 1, 2)?
 - 11) ¿Tienes un caso de prueba en el que todos los lados sean cero (0, 0, 0)?
 - 12) ¿Tienes al menos un caso de prueba que especifique valores no enteros (tales como 2.5, 3.5, 5.5)?
 - 13) ¿Tienes al menos un caso de prueba que especifique un número incorrecto de valores (dos en lugar de tres enteros, por ejemplo)?

4. Diseño de Casos de Prueba:

Id.	Descripción	Datos Entrada	Resultado Esperado
1	Triángulo escaleno válido	(3, 4, 5)	El triángulo es escaleno.
2	Triángulo equilátero válido	(7, 7, 7)	El triángulo es equilátero.
3	Triángulo isósceles válido	(5, 5, 8)	El triángulo es isósceles.
4a	Triángulo isósceles válido (Permutación 1: a=b)	(3, 3, 4)	El triángulo es isósceles.
4b	Triángulo isósceles válido (Permutación 2: a=c)	(3, 4, 3)	El triángulo es isósceles.
4c	Triángulo isósceles válido (Permutación 3: b=c)	(4, 3, 3)	El triángulo es isósceles.
5	Un lado con valor cero	(3, 0, 5)	Triángulo inválido: Las longitudes de los lados deben ser positivas.
6	Un lado con valor negativo	(3, 4, -5)	Triángulo inválido: Las longitudes de los lados deben ser positivas.
7	Suma de dos lados igual al tercero	(1, 2, 3)	Triángulo inválido: Las longitudes no cumplen la desigualdad triangular.
8a	Suma de dos lados igual al tercero (Permutación 1: a+b=c)	(1, 2, 3)	Triángulo inválido: Las longitudes no cumplen la desigualdad triangular.
8b	Suma de dos lados igual al tercero (Permutación 2: a+c=b)	(1, 3, 2)	Triángulo inválido: Las longitudes no cumplen la desigualdad triangular.
8c	Suma de dos lados igual al tercero (Permutación 3: b+c=a)	(3, 1, 2)	Triángulo inválido: Las longitudes no cumplen la desigualdad triangular.
9	Suma de dos lados menor que el tercero	(1, 2, 4)	Triángulo inválido: Las longitudes no cumplen la desigualdad triangular.
10a	Suma de dos lados menor que el tercero (Permutación 1: a+b<c)	(1, 2, 4)	Triángulo inválido: Las longitudes no cumplen la desigualdad triangular.
10b	Suma de dos lados menor que el tercero (Permutación 2: a+c<b)	(1, 4, 2)	Triángulo inválido: Las longitudes no cumplen la desigualdad triangular.
10c	Suma de dos lados menor que	(4, 1, 2)	Triángulo inválido: Las longitudes no

	el tercero (Permutación 3: $b+c < a$)		cumplen la desigualdad triangular.
11	Todos los lados son cero	(0, 0, 0)	Triángulo inválido: Las longitudes de los lados deben ser positivas.
12	Valores no enteros	(2.5, 3.5, 5.5)	Error: Por favor, ingrese solo números enteros para las longitudes de los lados.
13	Número incorrecto de valores (ej. solo 2)	(5, 5)	El comportamiento depende de cómo se maneje la entrada. El programa actual espera 3 entradas separadas. Si el usuario presiona Enter después de la segunda, esperará la tercera. Si se ingresan valores no numéricos, generará el error del caso 12.

- Los casos 1 a 4 prueban las clasificaciones válidas de triángulos.
- Los casos 5 a 11 prueban las condiciones de invalidez (lados no positivos y violación de la desigualdad triangular).
- El caso 12 prueba el manejo de errores para entradas no enteras.
- El caso 13 describe una situación que el código debe implementar de forma robusta.

6. En VS Code en el mismo proyecto “/LAB_01/” cree un nuevo archivo “triangle_test.py” y escriba el siguiente código Python:

```

"""
Este programa lee tres números enteros que representan las longitudes de los
lados
de un triángulo y determina si es equilátero, isósceles, escaleno o inválido.
"""

def clasificar_triángulo(lado1, lado2, lado3):
    """
    Clasifica un triángulo basado en las longitudes de sus lados.

    Args:
        lado1: Longitud del primer lado (entero).
        lado2: Longitud del segundo lado (entero).
        lado3: Longitud del tercer lado (entero).

    Returns:
        Una cadena de texto indicando el tipo de triángulo o si es inválido.
    """
    # Verificar que los lados sean positivos
    if lado1 <= 0 or lado2 <= 0 or lado3 <= 0:
        return "Triángulo inválido: Las longitudes de los lados deben ser
positivas."

    # Verificar la desigualdad del triángulo: la suma de dos lados debe ser
mayor que el tercero

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 12

```

    if (lado1 + lado2 <= lado3) or \
        (lado1 + lado3 <= lado2) or \
        (lado2 + lado3 <= lado1):
        return "Triángulo inválido: Las longitudes no cumplen la desigualdad
triangular."

# Clasificar el triángulo si es válido
if lado1 == lado2 == lado3:
    return "El triángulo es equilátero."
elif lado1 == lado2 or lado1 == lado3 or lado2 == lado3:
    return "El triángulo es isósceles."
else:
    return "El triángulo es escaleno."

# --- Programa Principal ---
if __name__ == "__main__":
    print("Ingrese las longitudes de los tres lados del triángulo:")

    try:
        # Leer las entradas del usuario y convertirlas a entero
        s1 = int(input("Lado 1: "))
        s2 = int(input("Lado 2: "))
        s3 = int(input("Lado 3: "))

        # Llamar a la función para clasificar y obtener el resultado
        resultado = clasificar_triángulo(s1, s2, s3)

        # Imprimir el resultado
        print(resultado)

    except ValueError:
        # Manejar el caso en que la entrada no sea un número entero válido
        print("Error: Por favor, ingrese solo números enteros para las longitudes
de los lados.")

```

7. Validar que cada caso de prueba diseñado en el punto 5. ¿El programa presenta algún error?

I. EJERCICIOS/PROBLEMAS PROPUESTOS

Para cada uno de los siguientes ejercicios:

- a) Cree el programa en Python
- b) Depurar y diseñar los casos de prueba necesarios para lograr el objetivo
- c) Valide que su programa ha pasado los casos de prueba

1. Calcular el área de un rectángulo

Enunciado:

Escribe un programa que solicite al usuario ingresar la base y la altura de un rectángulo como números enteros o decimales. El programa debe calcular el área del rectángulo ($\text{Área} = \text{base} * \text{altura}$) e imprimir el resultado mostrando claramente cuál es la base, la altura y el área calculada.

Objetivo de prueba: Verificar que el programa calcula correctamente el área para diferentes valores de base

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 13

y altura (enteros, decimales, positivos).

El informe debe contener el código fuente:

- rectangle.py
- rectangle_test.py

2. Identificador de Números Pares e Impares en una Lista

Enunciado:

Crea un programa que primero solicite al usuario cuántos números desea ingresar. Luego, debe pedirle que ingrese cada uno de esos números enteros. Finalmente, el programa debe recorrer la lista de números ingresados e imprimir cada número junto con un mensaje indicando si es "par" o "impar".

Objetivo de prueba: Comprobar que el programa maneja correctamente la cantidad de números especificada, lee los números, e identifica y etiqueta correctamente cada número como par o impar (incluyendo el número cero y números negativos si se consideran).

El informe debe contener el código fuente:

- numbers.py
- numbers_test.py

3. Mini Simulador de Cajero Automático (ATM)

Enunciado:

Desarrolla un programa que simule las operaciones básicas de un cajero automático. El programa debe iniciar con un saldo fijo (por ejemplo, S/.1,000). Debe presentar un menú al usuario con las siguientes opciones:

1. Consultar Saldo
2. Depositar Dinero
3. Retirar Dinero
4. Salir

El programa debe permitir al usuario seleccionar una opción.

- Si elige Consultar Saldo, debe mostrar el saldo actual.
- Si elige Depositar Dinero, debe solicitar la cantidad a depositar (validar que sea un monto positivo) y actualizar el saldo.
- Si elige Retirar Dinero, debe solicitar la cantidad a retirar (validar que sea un monto positivo y que no exceda el saldo disponible) y actualizar el saldo.
- Si elige Salir, el programa debe terminar.

El menú debe mostrarse repetidamente hasta que el usuario elija la opción "Salir".

Objetivo de prueba: Verificar que todas las opciones del menú funcionan correctamente: la consulta muestra el saldo correcto, los depósitos incrementan el saldo adecuadamente, los retiros disminuyen el saldo (respetando el límite del saldo disponible y validando montos positivos), y la opción de salir termina el programa. Probar casos límite como intentar retirar más dinero del disponible o depositar/retirar cero o montos negativos.

El informe debe contener el código fuente:

- atm.py
- atm_test.py

II. CUESTIONARIO

1. ¿Cuál es el objetivo de la depuración de software?
2. ¿Cuál es el objetivo de las pruebas de software?
3. Investigue sobre el perfil que debería tener un analista de pruebas y un ingeniero de pruebas de software
4. Dado que las **pruebas exhaustivas** (probar todas las combinaciones posibles de entradas y condiciones) son impracticables para la mayoría del software real. De su opinión sobre los criterios, técnicas a utilizar para garantizar una alta confiabilidad del software.
5. ¿Cuál es la diferencia entre verificación y validación de software?

III. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

- [1] Andreas Spillner, "Software Testing Foundations", Editorial Rocky Nook, 2021, 5th Edition
 [2] Glenford J. Myers, "The Art of Software Testing", Editorial Jhon Wiley & Sons, 2012, 3th Edition

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:

*Ejercicios propuestos /
Cuestionario*

INSTRUMENTOS:

Rúbrica

CRITERIOS DE EVALUACIÓN

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (55%)	Insuficiente (30%)	No presenta (0%)
Claridad y completitud de la explicación de los ejercicios propuestos. (30 %)	Todos los ejercicios propuestos han sido explicados de forma muy clara y con la profundidad adecuada.	Los ejercicios propuestos han sido resueltos y explicados, pero de manera muy superficial o general.	Los ejercicios propuestos han sido resueltos, sin embargo, algunos de ellos no han sido explicados.	Los ejercicios han sido resueltos parcialmente y no tienen explicaciones de lo realizado.	No presentan los ejercicios resueltos, o la versión final del informe omite por completo esta sección.
Ejercicios Propuestos (Código fuente) (20 %)	Todos los ejercicios propuestos se ejecutan correctamente y devuelven los resultados esperados.	Los ejercicios propuestos han sido implementados pero con los resultados esperados parciales.	Por lo menos el 2/3 de los ejercicios propuestos han sido implementados con resultados	Por lo menos el 1/3 de los ejercicios propuestos han sido implementados con resultados	No se han implementado los ejercicios propuestos.
Cuestionario (30%)	Todas las respuestas son completas, correctas y	La mayoría de las respuestas son correctas, pero algunas	Algunas respuestas son correctas, pero incompletas o	La mayoría de las respuestas son escuetas y/o	No presenta el cuestionario.

	proporcionan explicaciones claras, incluyen citas.	carecen de la profundidad esperada, pocas citas.	con falta de claridad en las explicaciones, citas no relevantes.	incompletas, no hay evidencia de haber buscado información que las respalden, sin citas.	
Presentación del Informe (20%)	El informe está bien estructurado, con excelente redacción, sin errores	El informe está bien estructurado, con algunos errores menores de redacción	El informe presenta problemas en la organización o redacción, con varios errores	El informe está desorganizado, con múltiples errores de redacción y formato	No presenta el informe.